Building with the Claude API

From API key to evaluated, tool-using assistant — a working lesson

Prepared
25 August 2026

Language
Python 3.12.10 · `anthropic` 0.122.0

Model throughout
`claude-opus-5`

Verified against
The SDK actually installed on this machine, not documentation alone. Signatures and method names in this document were introspected before being written.

Six chapters, in the order they were worked through: securing a key, authenticating, holding a conversation, giving Claude real actions to take, designing the contracts around those actions, and proving any of it works. Chapter 7 records findings specific to this repository.

Contents

1. Getting and storing an API key

1.1 Why nobody can retrieve it for you

Anthropic displays a new API key exactly once, at the moment you create it. After that only a hashed fingerprint is stored, which cannot be reversed. This is a safety property, not an inconvenience: it means a leak of Anthropic's own records does not leak your key.

The practical consequence: if you do not have the key saved, it is gone. Create a new one and delete the old. This costs nothing and takes under a minute.

1.2 Storing it safely on Windows

1. Go to **console.anthropic.com** → **API Keys** → **Create Key**.
2. Name it after the machine it will live on — `paul-laptop-aug2026`. If the laptop is ever lost you then know exactly which key to revoke without breaking anything else.
3. Copy the key. It begins `sk-ant-` and is roughly 108 characters. Use the copy button rather than selecting by hand.
4. Press **Windows**, type `environment`, open **"Edit environment variables for your account."**
5. In the **top** box (*User variables*), click **New...**
6. Name: `ANTHROPIC_API_KEY`. Value: paste the key. OK, OK.
7. Close and reopen every terminal and fully quit VS Code.

> **WHY THE DIALOG, NOT POWERSHELL**
>
> Anything typed into PowerShell is written to a plain-text history file on disk. Setting the key by command would leave a copy behind indefinitely. The dialog writes to your account's registry instead — not a file that gets swept into a backup, a shared drive, or a commit.

> **WHY THE TOP BOX**
>
> **User variables** apply only to your login. **System variables** (bottom box) expose the key to every account on the machine. There is no upside to the wider scope for a personal key.

1.3 The three traps

Trap	What happens
Pasting with quote marks	Windows treats `"sk-ant-..."` as including the quotes. You get a 401 with no hint why. Paste the bare key.
Hardcoding in source	`Anthropic(api_key="sk-ant-...")` puts a live secret in a file that can be committed, shared, or backed up. Use the zero-argument constructor instead — see §2.2.
Saving a “backup” copy	A note, a Word doc, an email to yourself. If you want a backup, use a password manager. One authoritative location is the point.

> **IF IT LEAKS, DON'T DELIBERATE**
>
> Delete the key in the Console and create a new one. Thirty seconds, no cost. A key you are unsure about is a key you should replace. This applies to pasting it into a chat, a ticket, or a screenshot.

2. Authenticating to the API

2.1 The three required headers

Your program sends an ordinary HTTPS request to `api.anthropic.com`. Proof of identity travels in the request **headers** — the envelope of labels alongside the message body.

Header	Value	Purpose
`content-type`	`application/json`	The body is JSON
`x-api-key`	your key	**The authentication**
`anthropic-version`	`2023-06-01`	Which version of the API's rules to apply

> **ON THAT VERSION DATE**
>
> It is not your key's date and not today's date. It is a fixed contract-freeze marker: by sending `2023-06-01` you ask for the rules as defined then, so Anthropic can evolve the API without silently breaking your code. It is the same literal string for everyone, and it is required on every request.

```
curl https://api.anthropic.com/v1/messages \
  -H "content-type: application/json" \
  -H "x-api-key: $ANTHROPIC_API_KEY" \
  -H "anthropic-version: 2023-06-01" \
  -d '{"model":"claude-opus-5","max_tokens":1024,
       "messages":[{"role":"user","content":"Hello"}]}'
```

2.2 Two ways to prove identity

	API key	Login profile
What it is	A long-lived secret you store	A browser login storing a short-lived token
Header sent	`x-api-key: sk-ant-…`	`Authorization: Bearer <token>` plus `anthropic-beta: oauth-2025-04-20`
Setup	Create in Console, set env var	Install the `ant` CLI, run `ant auth login`
Expires	No — valid until deleted	Yes, and refreshes itself
Best for	Servers, scripts, unattended work	Development on your own machine

The profile route means no static secret in your environment at all, which is genuinely better for laptop use. Note that converting a raw `curl` from a key to a profile token is a *header change*, not a value swap —

OAuth tokens go on `Authorization: Bearer`, never on `x-api-key`, and `/v1/messages` additionally requires the beta header shown above.

Either way, your code looks identical, and this is the form to use:

```
import anthropic

client = anthropic.Anthropic()   # no arguments — this is correct
```

The empty parentheses are not an oversight. The SDK goes looking for credentials itself.

2.3 Credential resolution order

The SDK checks these in order, and **the first match wins**:

1. `ANTHROPIC_API_KEY` environment variable
2. `ANTHROPIC_AUTH_TOKEN` environment variable
3. A login profile from `ant auth login`
4. Workload identity federation variables (cloud servers)
5. The default profile saved on disk

> **THREE FAILURE MODES WORTH MEMORISING**
>
> **A forgotten key beats everything below it.** Set a key months ago, log in with a profile later, and requests still silently use the old key — billing whichever account it belongs to.
>
> **An empty key still wins.** `ANTHROPIC_API_KEY=""` does not mean "no key"; it claims slot 1 and authenticates with nothing. It must be genuinely deleted, not blanked.
>
> **Setting both slot 1 and slot 2 breaks it.** The SDK sends both and the API rejects the request outright.

The rule that avoids all three: have exactly one credential configured. Not zero, not two.

2.4 Reading the error

The SDK raises a distinct named exception per case, so you never have to guess:

```
try:
    response = client.messages.create(...)
except anthropic.AuthenticationError:      # 401
    ...
except anthropic.PermissionDeniedError:    # 403
    ...
except anthropic.NotFoundError:            # 404
    ...
except anthropic.RateLimitError:           # 429 — the SDK already retries these
    ...
except anthropic.APIConnectionError:       # network
    ...
```

Error	HTTP	Plain meaning
`AuthenticationError`	401	Key missing, mistyped, or deleted
`PermissionDeniedError`	403	Key is real but not allowed to do this — wrong workspace or missing permission
`NotFoundError`	404	Usually **not** auth — a misspelled model name
`RateLimitError`	429	Too fast; the SDK retries automatically (default 2 retries)

> **THE HOUR-WASTER**
>
> A misspelled model ID surfaces as 404 “not found,” which sends people hunting through their credentials. **Auth failures are 401 and 403. Nothing else.**

3. Multi-turn conversations

3.1 The API has no memory

Anthropic stores nothing between calls. Each request is wholly independent. When it seems Claude remembers something from three turns ago, that is only because **you resent the entire conversation**.

So a "conversation" is not a server-side object. It is a list you keep and grow, and resend in full every turn.

3.2 Anatomy of the messages list

```
messages = [
    {"role": "user",      "content": "My name is Paul."},          # turn 1 — me
    {"role": "assistant", "content": "Nice to meet you, Paul."},   # turn 1 — Claude
    {"role": "user",      "content": "What's my name?"},           # turn 2 — me
]
```

`role` is who is speaking, and normally only two values occur: `"user"` (you or your end user) and `"assistant"` (Claude).

> **THE PART THAT SURPRISES PEOPLE**
>
> The `assistant` entries are Claude's own past replies, handed back as *input*. You are not describing what Claude said — you are replaying it. Claude reads that transcript fresh and infers "I must have said that."
>
> A consequence: you can edit or fabricate those entries, and nothing verifies them. That is legitimate and useful — trimming an old reply to save tokens, or seeding a conversation with an example exchange. It also means an incorrect entry becomes something Claude now believes it said.

The system prompt is not in the list. It is a separate parameter — the job description, persistent, not part of the dialogue, never something Claude replies to:

```
response = client.messages.create(
    model="claude-opus-5",
    max_tokens=16000,
    system="You are a concise assistant.",   # standing instructions, separate
    messages=messages,                       # the back-and-forth
)
```

Working example

```
import anthropic

client = anthropic.Anthropic()

SYSTEM = "You are a concise, friendly assistant."
messages = []

def send(user_text: str) -> str:
    # 1. Add my new message to the running history
    messages.append({"role": "user", "content": user_text})

    # 2. Send the ENTIRE history — not just the new message
    response = client.messages.create(
        model="claude-opus-5",
        max_tokens=16000,
        system=SYSTEM,
        messages=messages,
    )

    # 3. Add Claude's reply back into the history, in full
    messages.append({"role": "assistant", "content": response.content})

    # 4. Pull out readable text for display
    return "".join(b.text for b in response.content if b.type == "text")


print(send("My name is Paul, and I work in data analytics."))
print(send("What field did I say I work in?"))   # knows — because step 2 resent turn 1
```

3.3 Why you append blocks, not text

`response.content` is **a list of blocks**, not a string. One reply from Opus 5 can contain several:

Block type	What it is
`text`	The visible prose
`thinking`	Claude's reasoning — on by default for Opus 5
`tool_use`	A request to call one of your tools
`compaction`	Bookkeeping for very long conversations

The tempting shortcut throws information away:

```
# Works for plain chat, but discards thinking and tool blocks
text = next(b.text for b in response.content if b.type == "text")
messages.append({"role": "assistant", "content": text})
```

The robust form passes the blocks straight back — the SDK handles them:

```
messages.append({"role": "assistant", "content": response.content})
```

This habit means your code does not quietly break the day you add tools, or the day the conversation gets long enough for compaction to engage — compaction returns a block the API needs handed back, and text-only appending loses it silently.

> **ALSO ON THE READING SIDE**
>
> `response.content[0].text` looks reasonable and works right up until block zero is a thinking block. Always filter on `b.type` first.

Formatting rules

Rule	Detail
First message must be `user`	A conversation cannot open with `assistant`
Roles usually alternate	user → assistant → user → assistant
Same role twice is legal	Consecutive `user` messages merge into one turn — useful when someone sends two messages before Claude replies
`content` takes a string *or* a list	A plain string is shorthand; use a block list to mix text with images or documents
Order is meaning	The list is chronological. Reordering rewrites history

The richer `content` shape:

```
{"role": "user", "content": [
    {"type": "image", "source": {"type": "url", "url": "https://example.com/chart.png"}},
    {"type": "text",  "text": "What trend does this show?"},
]}
```

3.4 The cost consequence

Resending everything means input grows every turn. Turn 20 pays for turns 1–19 again. Total spend across a conversation grows with roughly the *square* of its length.

Two levers. **Prompt caching** — cached input costs about a tenth:

```
response = client.messages.create(
    model="claude-opus-5",
    max_tokens=16000,
    cache_control={"type": "ephemeral"},   # caches the stable front of the request
    system=SYSTEM,
    messages=messages,
)
```

Caching is a **prefix match** — reuse stops at the first byte that changed. Keep volatile content (timestamps, per-request IDs) out of the front of the request. Verify with `response.usage.cache_read_input_tokens`; if that stays zero across turns, something in your prefix is changing every time.

Compaction — for conversations approaching the context window, the API can summarise older parts server-side. This is where appending full `response.content` stops being merely wise and becomes required.

One extra option on Opus 5

To inject an instruction mid-conversation without disturbing the cached prefix, append a `system` role into the messages list:

```
messages.append({"role": "user", "content": "Summarize that."})
messages.append({"role": "system", "content": "Terse mode: under 40 words."})
```

Supported on Opus 5 with no beta flag. Three constraints: it cannot be first, it must follow a `user` message, and it must either end the list or be followed by an assistant turn. This one is genuinely model-dependent — Sonnet 5 rejects it with a 400 — so if you may switch models, keep standing instructions in the top-level `system` parameter.

4. A workflow assistant that takes action

4.1 The rule that shapes the design

> **THE WHOLE DESIGN FOLLOWS FROM THIS**
>
> **Claude decides *which* action to take. Your Python does the action.**

Claude never touches the data. It picks a function name and arguments; ordinary deterministic code does the rest. This is what makes handing real actions to a model acceptable: every side effect lives in code you can read, test, and re-run, and the probabilistic part is confined to routing.

The worked example below manages a task queue in a local JSON file — runnable with no external credentials. The structure is the reusable part, not the task; §5 covers how to redraw it around a real workflow.

4.2 Storage, confirmation, and the three tools

```
"""workflow_assistant.py — a Claude-powered work-item queue."""

import json
import pathlib
import uuid

import anthropic
from anthropic import beta_tool

STORE = pathlib.Path("tasks.json")
MODEL = "claude-opus-5"

client = anthropic.Anthropic()


def _load() -> dict:
    if not STORE.exists():
        return {"tasks": []}
    return json.loads(STORE.read_text(encoding="utf-8"))


def _save(data: dict) -> None:
    # Write to a temp file, then rename. A rename is atomic, so an
    # interrupted run can never leave a half-written tasks.json behind.
    tmp = STORE.with_suffix(".json.tmp")
    tmp.write_text(json.dumps(data, indent=2), encoding="utf-8")
    tmp.replace(STORE)


def _confirm(action: str) -> bool:
    answer = input(f"\n  Claude wants to: {action}\n  Allow? [y/N] ").strip().lower()
    return answer == "y"
```

Write-then-rename in `_save` is not ceremony. Without it, a crash mid-write leaves corrupt JSON and the next run cannot start.

The read-only tool

```
@beta_tool
def list_tasks(status: str = "all") -> str:
    """List work items in the queue.

    Call this whenever the user asks what is outstanding or what is done.
    Also call it before creating a task, to check for an existing duplicate,
    and before completing one, to look up the correct id.

    Args:
        status: Which items to return — "open", "done", or "all".
    """
    tasks = _load()["tasks"]
    if status in ("open", "done"):
        tasks = [t for t in tasks if t["status"] == status]
    return json.dumps(tasks, indent=2) if tasks else "No matching tasks."
```

> **THE DOCSTRING IS THE API CONTRACT**
>
> `@beta_tool` generates the JSON schema from your signature and docstring. Type hints become types; the docstring becomes the description; `Args:` lines become per-parameter descriptions; defaults make parameters optional. **The docstring is not a comment — it is the only thing Claude reads when deciding whether to call your function.**

Note the description says *when to call it*, not just what it does. That is deliberate: recent Opus models reach for tools conservatively, and stating trigger conditions measurably improves whether the tool gets called when it should. "Lists tasks" would be a worse description despite being accurate.

The write tool

```
@beta_tool
def create_task(title: str, owner: str, priority: str = "medium") -> str:
    """Create a new work item in the queue.

    Call this when the user asks for something to be added, tracked, or
    assigned to someone. Check list_tasks first — never create a second
    copy of an item that already exists.

    Args:
        title: Short description of the work, e.g. "Fix the nightly ETL retry".
        owner: Who the item is assigned to.
        priority: One of "low", "medium", "high".
    """
    if priority not in ("low", "medium", "high"):
        return f"Error: priority must be low, medium, or high — got {priority!r}."

    data = _load()

    # Idempotency: same title + same owner IS the same task, not a second one.
    for t in data["tasks"]:
        if t["title"].strip().lower() == title.strip().lower() and t["owner"] == owner:
            return f"Task already exists (id {t['id']}); nothing was created."

    if not _confirm(f'create "{title}" for {owner} ({priority})'):
        return "User declined. The task was not created."

    task = {
        "id": uuid.uuid4().hex[:8],
        "title": title,
        "owner": owner,
        "priority": priority,
        "status": "open",
    }
    data["tasks"].append(task)
    _save(data)
    return f"Created task {task['id']}: {title} ({priority}, {owner})."
```

Three defences, in deliberate order — **validate, dedupe, then confirm**:

1. **Validation first.** `priority` is described as an enum, but a description is guidance, not a guarantee. Check it in code.
2. **Dedupe before the prompt.** If the task already exists the user is never asked — a duplicate request becomes a no-op rather than a prompt.
3. **Confirm last**, so you are only ever asked about work that is actually going to happen.

The state-change tool

```
@beta_tool
def complete_task(task_id: str) -> str:
    """Mark a work item as done.

    Args:
        task_id: The 8-character id of the task, as shown by list_tasks.
    """
    data = _load()
    for t in data["tasks"]:
        if t["id"] == task_id:
            if t["status"] == "done":
                return f"Task {task_id} was already done; nothing changed."
            if not _confirm(f'mark "{t["title"]}" as done'):
                return "User declined. The task was not changed."
            t["status"] = "done"
            _save(data)
            return f"Task {task_id} marked done."
    return f"Error: no task with id {task_id}. Call list_tasks for valid ids."
```

> **THAT LAST LINE DOES REAL WORK**
>
> When Claude invents an id — it occasionally will — the error string goes back into the conversation, Claude reads it, calls `list_tasks`, and retries with a real id. **Error messages are instructions to Claude, not just log output.** "Invalid id" leaves it stuck; naming the recovery path gets it unstuck.

4.3 The agent loop

```
SYSTEM = """You are a workflow assistant managing a work-item queue.

- Check list_tasks before creating anything, to avoid duplicates.
- Never invent a task id — read ids from list_tasks.
- If a request is ambiguous (no owner given, unclear which task), ask
  instead of guessing.
- After acting, state plainly what changed."""

TOOLS = [list_tasks, create_task, complete_task]


def run(user_input: str) -> None:
    runner = client.beta.messages.tool_runner(
        model=MODEL,
        max_tokens=16000,
        system=SYSTEM,
        tools=TOOLS,
        messages=[{"role": "user", "content": user_input}],
        max_iterations=10,   # hard stop — a bug can't spin forever
    )

    for message in runner:
        for block in message.content:
            if block.type == "text" and block.text.strip():
                print(block.text)
            elif block.type == "tool_use":
                print(f"  -> {block.name}({block.input})")


if __name__ == "__main__":
    import sys
    run(" ".join(sys.argv[1:]) or "What's on the queue?")
```

`tool_runner` is what saves you from hand-writing the loop. Without it you would manage this yourself: call the API, check `stop_reason`, extract `tool_use` blocks, run the matching function, append results with matching ids, call again, repeat. `max_iterations` is the circuit breaker — two tools that trigger each other would otherwise loop until you noticed.

Running it

```
python workflow_assistant.py "add a task for Ali to fix the nightly ETL retry, high priority"
python workflow_assistant.py "what's still open?"
python workflow_assistant.py "add a task for Ali to fix the nightly ETL retry"   # deduped
```

The third is the one to actually try: a differently-worded duplicate that should return “already exists” without prompting. Then try `"mark the ETL one done"` and watch Claude call `list_tasks` first to find the id. You never asked for two steps — the tool descriptions produced them.

TWO HONEST LIMITATIONS

Single-shot. Each invocation starts fresh. To make it conversational, keep a `messages` list between calls as in §3.

No `is_error` flag. When a `@beta_tool` returns a string, that string is the result; there is no separate error marker. Claude handles descriptive error strings fine — which is why their wording carries weight. If you specifically need the flag, that requires the manual loop.

5. Tool schemas and result handling

5.1 Schema design decisions

Granularity — the decision that matters most

Before parameters or descriptions, decide how many tools and where the seams go.

Shape	Example	Result
Too coarse	`manage_task(action, **kwargs)`	Claude must guess which fields pair with which action. Errors move from "wrong tool" to "wrong arguments" — harder to catch
Right	`list_tasks`, `create_task`, `complete_task`	Each schema is exactly right for one job
Too fine	`set_task_priority`, `set_task_owner`, …	Claude burns turns chaining; the tool list crowds out reasoning

The heuristic: **one tool per thing a user would ask for.** Nobody asks to "set task priority" in isolation — they ask to create a task, priority included. That is the seam. Keep the set small; a focused four beats a vague twelve.

Required vs optional is a real decision

```
def list_tasks(status: str = "all") -> str:      # optional — Claude may omit it
def create_task(title: str, owner: str) -> str:  # required — must supply both
```

Optional means Claude will sometimes leave it out and your default applies silently. Make required exactly the things with no sane default, where guessing wrong is worse than asking.

`owner` is the interesting case. Required means Claude must extract it or ask. Optional with `owner: str = "unassigned"` means "add a task to fix the ETL" quietly creates an unowned task. Neither is wrong — but that is a workflow policy decision being made in a type signature, so make it deliberately.

Validate anyway

Naming an enum in the schema both constrains generation and documents itself. You still check in code, because a description shapes behaviour rather than guaranteeing it:

```
if priority not in ("low", "medium", "high"):
    return f"Error: priority must be low, medium, or high — got {priority!r}."
```

When to abandon the decorator

Hand-write raw JSON schema when the signature cannot express what you need:

- **Nested objects or arrays of objects** — `line_items: [{sku, qty}]`
- **Runtime-generated tools** — schemas built from config or a database
- `strict: True` — a guarantee that `tool_use.input` validates exactly, rather than usually

```
tools = [{
    "name": "create_task",
    "description": "Create a work item. Call this when the user asks for "
                   "something to be tracked or assigned.",
    "strict": True,
    "input_schema": {
        "type": "object",
        "properties": {
            "title":    {"type": "string", "description": "Short description of the work"},
            "owner":    {"type": "string", "description": "Who it's assigned to"},
            "priority": {"type": "string", "enum": ["low", "medium", "high"]},
        },
        "required": ["title", "owner", "priority"],
        "additionalProperties": False,   # required for strict
    },
}]
```

`strict` needs both `required` and `additionalProperties: False`. Worth it when a malformed input is expensive — a payment, an outbound email, a production write. For a local JSON file it is overkill.

5.2 Results as messages into context

> **THE REFRAME**
>
> A tool's return value is not a return value in the ordinary sense. **It is a message you are writing into Claude's context.** One string serves three purposes at once: Claude's only evidence of what happened, the input to its next decision, and the basis of what it tells your user.

Rule 1 — Return the resulting state, not an acknowledgement

```
return "OK"                                                  # useless
return f"Created task {task['id']}: {title} ({priority}, {owner})."   # useful
```

`"OK"` forces Claude to describe an outcome it cannot see — exactly when models fill gaps with plausible invention.

Rule 2 — Return the identifiers needed for the next step

Claude cannot call `complete_task(task_id=...)` unless an earlier result contained that id. **Chaining is only possible through returned values.** When a multi-step workflow stalls, the usual cause is a tool that did its job but returned nothing consumable.

Rule 3 — Budget the size

Tool results are conversation history, so **every result is resent on every subsequent turn**. A tool that dumps 50 KB of JSON does not cost you once — it costs you every turn for the rest of the session, and

crowds out the reasoning space that made the assistant useful. Paginate and summarise *at the tool boundary*:

```
@beta_tool
def list_tasks(status: str = "all", limit: int = 20) -> str:
    """List work items in the queue.
    ...
        limit: Maximum items to return. Defaults to 20.
    """
    tasks = _load()["tasks"]
    if status in ("open", "done"):
        tasks = [t for t in tasks if t["status"] == status]

    total = len(tasks)
    shown = tasks[:limit]
    # Return only fields Claude needs to act — not the whole record
    slim = [{"id": t["id"], "title": t["title"], "owner": t["owner"],
             "priority": t["priority"]} for t in shown]

    out = json.dumps(slim, indent=2)
    if total > limit:
        out += f"\n\n({limit} of {total} shown. Narrow with status, or raise limit.)"
    return out
```

> **WHY THE TRUNCATION NOTE MATTERS**
>
> Without it, Claude treats a partial list as complete and will confidently tell your user there are 20 tasks when there are 400.

Rule 4 — Nothing sensitive in a return value

Results land in Claude's context, in your logs, and in the transcript. Return `"Emailed a******@example.com"` rather than the full address; `"Auth succeeded"` rather than the token. If a tool reads a table with PII, project only the columns you need.

Rule 5 — Errors are instructions

Instead of	Write
`"Error: not found"`	`"No task with id a1b2c3. Call list_tasks for valid ids."`
`"Invalid input"`	`"priority must be low, medium, or high — got 'urgent'."`
`"Request failed"`	`"Upstream timed out after 10s. Safe to retry once."`

The last one carries information Claude cannot otherwise have: whether retrying is safe. For anything with a side effect, say so explicitly — otherwise a retry decision is a guess.

Rule 6 — Report partial success honestly

```
return (f"Created {len(created)} of {len(requested)}. "
        f"Created: {', '.join(created)}. "
        f"Failed: {failed_detail}")
```

Claude relays what you return. Vague returns become confident, wrong summaries.

5.3 The mechanical layer

Worth understanding even though the runner handles it, because it is what you will debug. Claude's response carries a `tool_use` block with `id`, `name`, `input`. Your reply must contain a `tool_result` whose `tool_use_id` matches that id exactly.

```
for block in response.content:
    if block.type == "tool_use":
        result = execute_tool(block.name, block.input)
        tool_results.append({
            "type": "tool_result",
            "tool_use_id": block.id,      # must match, or the API rejects it
            "content": result,
            # "is_error": True            # only available via the manual loop
        })

messages.append({"role": "assistant", "content": response.content})  # full content
messages.append({"role": "user", "content": tool_results})            # ALL results, one message
```

Three failure modes live in those lines:

- **Parallel calls must return in a single user message.** Claude often requests two or three tools at once. Splitting the results does not error — it silently teaches Claude to stop parallelising, and your assistant gets slower for reasons that never appear in a log.
- **Append the full `response.content`.** Text-only appending drops the `tool_use` blocks, and the `tool_result` you send next then references an id no longer present in history.
- **Parse `input` as JSON; never string-match it.** Opus 5 varies its JSON string escaping. `json.loads()` handles it; a regex over the serialised form breaks intermittently — the worst way for anything to break.

5.4 Intercepting without leaving the runner

You do not need the manual loop for control. Each iteration hands you the assistant message **before** your tools run:

```
for message in runner:
    # 1. Inspect what Claude wants to do, before it happens
    for block in message.content:
        if block.type == "tool_use":
            log_intent(block.name, block.input)

    # 2. Inspect the result before it goes back to Claude.
    #    Cached — your tools still execute exactly once.
    response = runner.generate_tool_call_response()
    if response is not None:
        audit(response)
```

Method	Use
`generate_tool_call_response()`	Read or modify the result before Claude sees it. Cached — tools run once
`set_messages_params()`	Override the pending request — deny an action, inject context
`append_messages()`	Push extra messages into the conversation mid-loop
`until_done()`	Run the whole loop in one call when you need no hooks

Your tools execute automatically **only if you do not intervene**. So a gate can live inside the tool function (simple) or out here (better when the policy is cross-cutting — every write requires approval, and you would rather not duplicate that check in six functions).

Return contracts by tool class

Tool class	Confirm?	Return should contain
Read — queries, listings	No	Slim projection, ids, explicit truncation note
Write — create, update	Yes	New state, id, and *whether anything actually changed*
External call — email, HTTP	Yes	Outcome, whether retry is safe, redacted identifiers
Destructive — delete, refund	Always	What was removed, and how to reverse it if possible

> **THE ROW PEOPLE LEAVE OUT**
>
> "Whether anything actually changed." `"Task already exists; nothing created"` and `"Created task a1b2c3"` must be distinguishable, or Claude reports a duplicate request as a fresh success — and your idempotency check becomes invisible to the user it was protecting.

6. Evaluating prompts objectively

6.1 Dataset design

An eval dataset is a JSONL file — one JSON object per line, no commas between lines. Each case carries something to send and something to check:

```
{"input": {"message": "we are down"}, "expected": {"urgency": "HIGH"}}
{"input": {"message": "quick question"}, "expected": {"urgency": "LOW"}}
```

The categories a good set covers

Category	Why it's there
Clear ends of the scale	Baseline — if these fail, everything is broken
Multiple issues in one input	Tests the tie-break rule
Category collision	Tests the override rule (e.g. technical fault *and* billing complaint)
Too vague to place	Tests the documented default
Out-of-band input	An auto-reply, a bare "thanks" — tests that it declines to act
Prompt injection	"IGNORE ALL PREVIOUS INSTRUCTIONS" embedded in the data — tests that data stays data
Degenerate input	Empty string — tests it doesn't invent a request

> **GRADE ONLY WHAT THE CASE IS ABOUT**
>
> If a case exists to test owner routing and the urgency is genuinely arguable, assert *only* the owner. Grading fields you are uncertain about bakes your own uncertainty into the score as noise.

Four things that are usually missing

- **Volume.** Ten cases means each is worth 0.10 — you cannot tell a real improvement from one case flipping. For a multi-way classification, ~40–50 cases is where the number starts holding still, weighted toward boundaries rather than obvious cases.
- **Coverage of every output field.** It is common to grade two of four required fields and never notice. Some free-text constraints are mechanically checkable without a judge:

  ```
  len(summary.split()) <= 20
  re.search(r"\b(today|24 hours|by \d|within \d+ ?(min|hour|day))", next_action, re.I)
  ```

- **A held-out split.** Iterating prompt versions against the same cases is fitting to the test set. Keep a portion you touch only to confirm.

- **Labelling discipline.** Write the expected value from the *rule*, not from what the model produced. The failure mode is subtle: you disagree with a label, look at the output, decide the model's answer was reasonable, and update the expectation. Do that a few times and your eval measures agreement with the model rather than correctness.

Datasets for tool-using assistants

An agent needs a different notion of correct. Extend the same JSONL shape with new keys:

```
{"input": "add a task for Ali to fix the nightly ETL, high priority",
 "expect_tools": ["list_tasks", "create_task"],
 "forbid_tools": ["complete_task"],
 "expect_state": [{"title_contains": "ETL", "owner": "Ali",
                   "priority": "high", "status": "open"}]}

{"input": "add a task for Ali to fix the nightly ETL",
 "seed_state": [{"id": "aaa11111", "title": "Fix nightly ETL",
                 "owner": "Ali", "priority": "high", "status": "open"}],
 "expect_state_unchanged": true,
 "expect_no_confirmation_prompt": true}
```

Dimension	Assertion
Tool selection	Tool names called match `expect_tools`
Argument correctness	`create_task` received the right owner and priority
Restraint	Nothing in `forbid_tools` was called
End state	The data file matches `expect_state`
Idempotency	Re-running leaves state unchanged and prompts nobody

> **GRADE THE END STATE, NOT THE PATH**
>
> For an agent you do not need to grade reasoning or trajectory. The objective truth is what the system looks like afterward — a deterministic assertion, no judge, no ambiguity.

Two implementation notes: capture the trajectory by recording `block.name` and `block.input` for each `tool_use` block as you iterate the runner; and stub `_confirm()` to auto-approve during evals, or every case blocks on `input()` forever.

6.2 Grading in three tiers

Use the cheapest tier that can answer the question. Most people reach for tier 2 when tier 1 would do.

Tier 1 — Deterministic (default)

Exact match, numeric tolerance, regex, set comparison, end-state assertion. Free, instant, perfectly repeatable. **Everything with a finite set of correct answers belongs here** — labels, enums, ids, counts, word limits, state.

Tier 2 — LLM judge (open text only)

Use structured outputs so the verdict is parseable rather than prose:

```
from pydantic import BaseModel

class Verdict(BaseModel):
    passes: bool
    reason: str          # one sentence, cited to the rule

RUBRIC = """You are grading one field of a record against its rule.

Rule: {rule}
Field value produced: {value}
Source input: {source}

Pass only if the value satisfies the rule. Judge the rule as written — not
whether you would have phrased it differently. If uncertain, fail it."""

verdict = client.messages.parse(
    model="claude-opus-5",
    max_tokens=2000,
    messages=[{"role": "user", "content": RUBRIC.format(...)}],
    output_format=Verdict,
).parsed_output
```

Four rules for judges:

- **One dimension per call.** “Accurate, concise, and free of invented facts?” gives a blurred average. Three calls give three signals — and tell you which one broke.
- **Do not show the judge a gold answer** when grading open text. It anchors on wording and penalises valid alternatives. Show it the *rule*.
- **Bias it toward failing** when uncertain. An optimistic judge inflates every score equally, hiding regressions.
- **Calibrate before trusting.** Hand-label 20 cases, run the judge, measure agreement. Below ~90% the judge is a noise source, not a measurement. This step is almost universally skipped, and it is the difference between a metric and a vibe.

Tier 3 — Human spot-check

Read ten outputs yourself after every prompt change. Not for scoring — for noticing the failure mode you never wrote a case for. No automated tier invents the prompt-injection case; a human reading outputs does.

6.3 Making the number trustworthy

Change	Why
Run each case N times and report mean plus min–max	Highest-value change available. Without it you cannot distinguish a real improvement from a coin flip. On 10 cases, a 0.10 difference *is one case* — treat it as noise
Per-field accuracy and a confusion matrix	Keep strict pass/fail per case, but add the breakdown. Direction of error is the whole story
Persist every run to a JSONL	Prompt path, version, model, effort, date, per-case results. Otherwise "did v1.1 help?" is unanswerable afterward
Turn the effort down	Cuts cost and latency substantially on fixed-format classification work

```
urgency accuracy: 8/10          owner accuracy: 9/10
  expected HIGH   -> got MEDIUM  x1     <- a missed outage
  expected MEDIUM -> got LOW     x1
```

Those two errors cost the same 0.10 and mean completely different things operationally. One is a customer waiting on a stopped system. Aggregate scores hide that.

```
response = client.messages.create(
    model=MODEL,
    max_tokens=MAX_TOKENS,
    output_config={"effort": "low"},
    messages=[{"role": "user", "content": filled_prompt}],
)
```

> **LOWER THE EFFORT; DON'T DISABLE THINKING**
>
> Thinking is on by default on Opus 5, and turning it off has a known side effect of leaking internal tags into the visible response — which would land straight in whatever regex parses your output. Effort is the safe lever. Record it in the run log: a score is only comparable to another score at the same effort.

7. Findings in this repository

Recorded 25 August 2026 against `scripts/score_prompt.py`, `evals/triage.jsonl`, and the three prompt files.

7.1 The documented command scores a stale prompt

File	Lines	Has `NONE`	Awkward-situations block
`prompts/triage.md`	30	**No**	**No**
`prompts/triage/v1.0.0.md`	74	Yes	Yes
`prompts/triage/v1.1.0.md`	79	Yes	Yes

`prompts/triage.md` is a stale copy predating v1.0.0; it caps urgency at "HIGH, MEDIUM, or LOW." The command documented at `score_prompt.py` line 12 points at exactly that file. Cases 7 (auto-reply) and 9 (empty message) both expect `"urgency": "NONE"` — a value the stale prompt cannot emit.

> **EFFECT**
>
> The documented command has a hard ceiling of **0.80**, and two of ten cases fail for a reason unrelated to prompt quality.

Fix: point the documented command at `prompts/triage/v1.1.0.md` and delete the unversioned copy. Explicit versions mean you always know what you scored; the drift already happened once, which is the argument against keeping a second source of truth.

7.2 Half the output contract is untested

The prompt requires four fields — `Urgency`, `Summary`, `Next action`, `Owner`. The eval set checks only urgency and owner. `Summary` (≤20 words) and `Next action` (must include a time commitment) have no coverage at all, and both are partly checkable deterministically (see §6.1).

7.3 Single-run scores on a ten-case set

Each case runs once. The script's own docstring correctly notes that Claude is not perfectly repeatable, then reports a single number as though it were. One flip moves the score 0.10 — larger than most genuine prompt improvements.

7.4 Priority order

1. Repoint the documented command at a versioned prompt; delete `prompts/triage.md`. Recovers the 0.20 ceiling. Five minutes.
2. Add `--runs N` with mean and range, plus per-field accuracy and an urgency confusion matrix.
3. Add the two deterministic checks for `Summary` word count and `Next action` time commitment.

4. Grow the dataset toward 40 cases, keeping ten held out.
5. Add the tool-trajectory format, starting with the idempotency case.

Items 1–3 touch `scripts/`, so this repository's `PROGRESS.md` gate applies: session ID, entry with verification evidence, regenerated session changelog.

8. Quick reference

Models

Pricing per million tokens. Figures cached mid-2026 — confirm current rates before relying on them for budgeting.

Model	ID	Context	Input	Output
Claude Fable 5	`claude-fable-5`	1M	$10.00	$50.00
Claude Opus 5	`claude-opus-5`	1M	$5.00	$25.00
Claude Sonnet 5	`claude-sonnet-5`	1M	$3.00	$15.00
Claude Haiku 4.5	`claude-haiku-4-5`	200K	$1.00	$5.00

Use the exact ID strings — never append date suffixes. Default to Opus 5 unless you have a specific reason; do not downgrade purely for cost without deciding that deliberately.

Parameters worth knowing

Parameter	Note
`max_tokens`	A ceiling, not an instruction — unused capacity costs nothing. Default ~16000 non-streaming, ~64000 streaming. Hitting the cap truncates mid-thought
`output_config={"effort": ...}`	`low` … `max`; default `high`. Nested inside `output_config`, not top-level
`thinking={"type": "adaptive"}`	On by default on Opus 5. Add `"display": "summarized"` to see the reasoning; the default returns empty thinking text
`cache_control={"type": "ephemeral"}`	Prefix match. Minimum cacheable prefix ~1024 tokens; shorter silently won't cache
`timeout` / `max_retries`	Default 10 minutes and 2 retries. Python takes *seconds*; TypeScript takes milliseconds
`max_iterations`	On `tool_runner` — your circuit breaker against a self-triggering tool loop

Mistakes to avoid

- Hardcoding a key instead of using the zero-argument client constructor
- Reading `response.content[0].text` instead of filtering on `block.type`
- Appending only text to history, dropping thinking and `tool_use` blocks
- Splitting parallel `tool_result` blocks across multiple messages
- String-matching a tool's serialised `input` instead of `json.loads()`

- Returning `"OK"` from a tool instead of the resulting state
- Letting a tool dump unbounded output into context
- Reporting a single eval run as though it were a stable measurement
- Trusting an LLM judge that has never been calibrated against human labels

---

Every code sample here was written against `anthropic` 0.122.0 on Python 3.12.10, with SDK signatures and runner method names introspected from the installed package rather than recalled. Where behaviour is model-dependent or version-dependent, that is stated inline.